EPFL - Computer Science

COM-402 Information Security and Privacy

Lecture 02 — September 23, 2025

Author: Jonathan Dutoit
Professor: Mathias Payer

Crypto Basics

Learning Goals

- Basic of symmetric crypto (stream ciphers, block ciphers, operation modes)
- Data integrity (hash functions, message authentication codes)
- Authenticated Encryption
- Public-key Cryptography (Digital signature, Diffie-Hellman key exchange)
- Public Key Infrastructure (PKI)
- Case study: TLS

Goals of Cryptography

- **Confidentiality**: Data is only accessible with the correct key
- **Integrity**: Any modification of data can be detected
- **Authentication**: The author of a message can be identified
- **Non Repudiation**: The author of a message cannot deny being the author

Examples of Cryptographic Schemes

- Caesar's
- Vigenere
- Elliptic-Curve Crypto
- RSA
- Enigma

  Used during the second World War, cryptography became a very important tool. Enigma is one of the prime example which was thought to be secure, multi-dimensional rotation used by the Germans, broken by the British and Americans and used to intercept and decipher the messages. Weakness:

  - Beginning was always the weather or "Hi Hitler" which made it susceptible to known plain text attacks. Allowed the team to test if they correctly decoded the message (quick sanitization)
  - An A would never be encoded to another A due to the mechanical property of the multi-dimensional rotation.

Brief Look into the Past

- -50 Caesar's Cipher (substitution)
- ...
- Kerckhoffs' principle
- 1970s Data Encryption Standard (DES)
- 1970s Diffie-Hellman public key crypto
- 1970s Rivest, Shamir, Adleman RSA
- 1985 Elliptic curve crypto (ECC)
- 1990s MD5, SHA-1 hash algorithms
- 2001 SHA-2
- 2001 Advanced Encryption Standard (AES)
- 2015 SHA-3

Brief Look at the Present/Future

If quantum computers ever come to exist, they will break many crypto algorithms (mainly asymmetric).

Current research: Post-quantum crypto:

more complicated but plausibly unbreakable by quantum computers

Follow up from professor's view: likes the idea of quantum computers because they are extremely powerful, but will this ever materialize? Will we have strong enough quantum computer to do actually useful things?

As a student mentioned, we already have a quantum computer that have a couple of cubits but they are extremely expensive and limited.

Can we see a path were we can extend the number of cubits towards where we can do some actually useful computation such as breaking cryptography?

Personal opinion: a bit sceptic about it. We'll see, but it's definitely an interesting playground for cybersecurity.

Naive Approach to Encryption

Two parties agree on an algorithm (e.g., rot13) and keep it secret. Caesar's cipher was as simple shift of letters.

If you know that it is a shift, it only takes 26 trials to break it (i.e., susceptible to brute force)

1883 - Kerckhoffs's principle

A cryptosystem should be secure even if everything about the system, except the key, is public knowledge.

One-Time Pad

Key is a random string at least as long as the plaintext.

- XOR operation to encrypt and decrypt
- $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$

Encryption: $c = m \oplus key$

Decryption: $m = c \oplus key$

One-time pad is **perfect** in theory.

Practical disadvantages:

- Requires truly uniform random one-time pad values
- Key **must not** be used more than once. If you have multiple ciphertexts, you can combine them, either getting the key or the message.
- Key length depends on the message length
  - e.g., it needs a 1TB key to encrypt a 1TB disk

So, it's perfectly secure but impractical. For all of the crypto systems that we look at, we want to get as close to the one-time pad as possible, but being more practical.

One-Time Pad Discussion

Why use **XOR** and not **addition**?

- I.e. ciphertext = plaintext + key
- Potential information leaks! (violates confidentiality.

Ex: messages and keys chosen from the space of positive integers

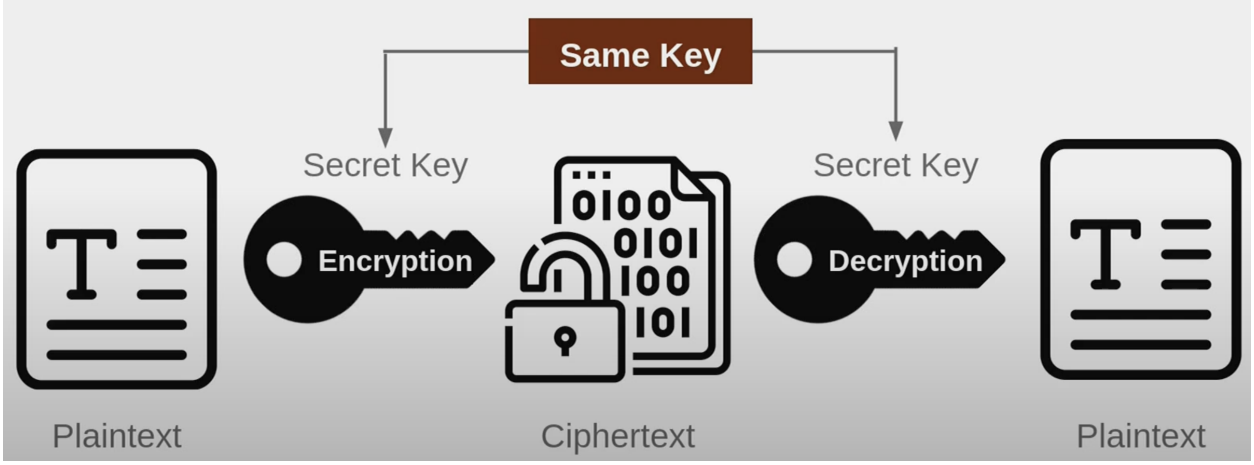
- Let plaintext = 3, key = 10
- Ciphertext = 3 + 10 = 13
- Adversary can infer that plaintext ¡ 13 !

Symmetric Crypto

Encryption and Decryption are done with the **same key** hence symmetric.

Solves the problem of transferring large amounts of confidential data

Reduces security to the problem of transferring a symmetric key (**key exchange**)



Stream Cipher

Use Pseudo Random Generator to expand a short seed into a stream of random bits

$$\{0,1\}^k \Rightarrow \{0,1\}^n, n > k.$$

- n-k bit stretch
- Use part of output as new seed
- Can encrypt data of arbitrary length
- Stream Cipher: XOR message and pseudorandom pad
- E.g., Salsa20, Chacha20

Limitation of Stream Ciphers

Two possible pitfalls with stream ciphers:

Malleability: Flipping one bit of the ciphertext will flip the same bit in the plaintext

- e.g., if you know which bit encodes the sign of a value you can change a payment of $100 to -100$
- you should always add an **integrity check** when encrypting data!

Cipher reuse: if two cleartexts are encrypted with the same cipher-stream, then:

$M_1 \oplus M_2 = C_1 \oplus C_2$

- If you know bits of one message, you can deduce the corresponding bits of the other message: $M_2[k] = M_1[k] \oplus C_1[k] \oplus C_2[k]$
- You should always use unique **seed** to create a pseudorandom cipher-stream

Block Cipher

Encrypts **fixed size** blocks of data

- a **padding scheme** is used to fill the last block
- a **mode of operation** to combine multiple blocks

Data Encryption Standard (**DES**):

- block size 64 bits (too short, collisions)
- key size 56 bits (too short, can be brute forced)

Don't use DES (or triple DES), both are deprecated

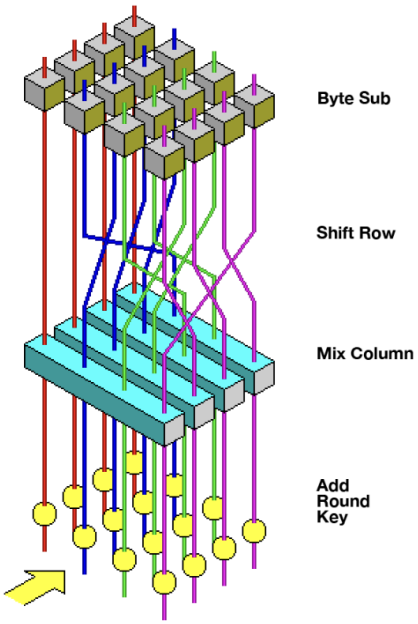
Advanced Encryption Standard (**AES**):

- block size 128 bits
- key size 128/192/256 bits
- hardware support (e.g., Intel, AMD, ARMv8)

When in doubt, use AES

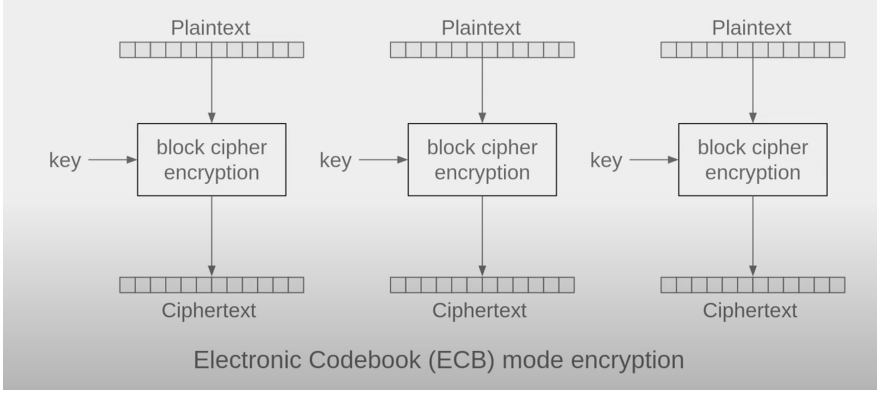
Block Cipher: AES (Rijndael)

- A block cipher shuffles input on a per-block basis
- Each output byte depends on all input bytes
- Multiple rounds ensure thorough shuffling

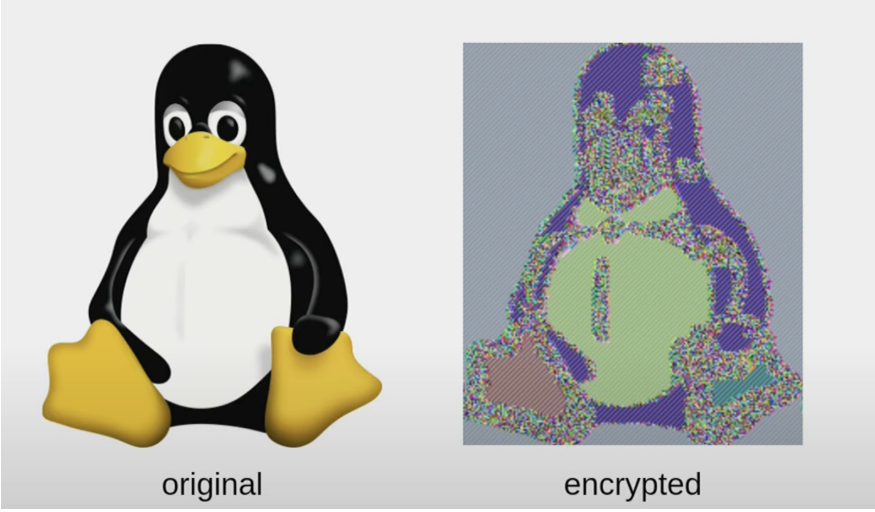


Modes of Operation: ECB

ECB: Encrypt each block separately, with the same key



Problem: You can see patterns in the ciphertext. If you have a colored imaged, you can see the shape:



Modes of Operation: CBC

Introduces the use of an Initialization Vector (IV) for the first block

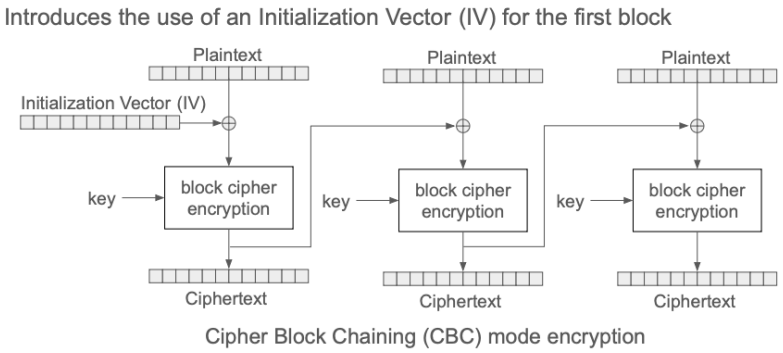


Figure 1: Cipher Block Chaining (CBC) mode encryption

Each ciphertext block acts as IV of the next block.

Decryption is the opposite of encryption.

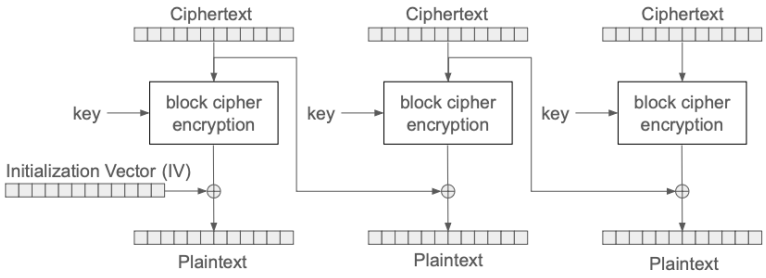


Figure 2: Cipher Block Chaining (CBC) mode decryption

CBC encryption does not reveal any structure

You can no longer see the penguin...

Quiz!

What happens to decryption if a bit is a flipped due to a transmission error?

- does the error propagate to all following blocks?
- does it only affect the current block?
- none of the above?

Answer: A whole block is garbled, one bit of the next block is inverted.

Some possible pitfalls in using CBC:

Malleability:

- Flipping one bit of the IV will flip the same bit in the plaintext
- Flipping one bit in a ciphertext block flips the same bit in the next plaintext block and mangles the current block
- **Always** add an **integrity check** when encrypting data!

Padding oracle attacks:

- The last block must be padded to obtain the correct block size
- If not carefully implemented, validation of padding can lead to leakage of the plaintext (e.g., in TLS, see Serge Maude nay's BCC padding oracle attack)

Data Integrity

Data Integrity

Integrity: protects against unauthorized modification

integrity $\neq confidentiality$

Remember the malleability of stream and block ciphers: **it's easy to flip bits of the cleartext**

We need a primitive that:

- detects any modification of the message
- cannot be forged (circumvented) by an attacker

(Cryptographic) Hash Functions

Goal: Take an arbitrary length input, and generate fixed length output

Cryptographers say they map a 'preimage' to an 'image'

Properties of Hash Functions

Preimage resistance:

- Given a hash h, it is difficult to find a message m for which h = hash(m)
- This implies that the function is a one-way function (and P != NP)
- Very useful for password hashing

Prevented attack: given a password hash, an attacker cannot recover the **matching** password.

Second preimage resistance:

- Given a message m_1, it is difficult to find a second message m_2 such that $hash(m_1) = hash(m_2)$

Prevented attack: hashes protect integrity of messages, thus, an attacker cannot forge a message with the same hash as an existing message.

Collision resistance

- It is difficult to find any two messages m_1, m_2 such that $hash(m_1) = hash(m_2)$

Possible attack: hashes are used in signatures. An attacker could show one preimage and have signed by a third party. The signature would also be valid for the other preimage.

Quiz!

What is the complexity of randomly finding a 2nd preimage in a random function that generates 160 bit outputs (e.g., SHA-1)?

What is the complexity of randomly finding a collision in SHA-1?

NB: if the hash function has some weaknesses, the complexity might be lower.

Answer:

Collisions in naively: 2^{60} operations
better: 2^{80} operations
Hashing 2^{80} random messages would produce one expected collision based on the birthday paradox.

In practice, it took $2^{63.1}$ operations to find a preimage.

Example of Hash Functions

MD5: 128 bits, chosen prefix collisions are easy (do not use)

SHA-1: 160 bits, broken, collisions are difficult but possible (avoid)

SHA-2: 224/256/384/512 bits, not broken, but related to SHA-1 (use)

SHA-3: 224/256/384/512 bits, no weakness know(prefer)

BLAKE3: 224/256/384/512 bits, no weakness know, faster, not standard (fine)

Key takeway: complex enough to be hard to brute force but reasonable to compute. Which means that even on today hardware, we need to able to computer. Even on embedded systems too.

Message Authentication Codes (MAC)

Similar to a hash function, but involves a symmetric key

The same key is used to generate the MAC and to validate it

If they key is know only to the two parties of an exchange, a correct MAC proves:

- that the messages was not created by a third party (**authentication**)
- that the message has not been modified (**integrity**)

Typically based on a hash function (e.g. HMAC SHA-2) or on block ciphers (e.g. CBC-MAC-AES)

Authenticated Encryption

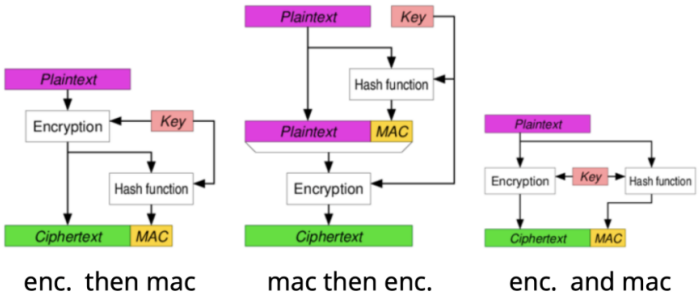
Confidentiality and Integrity

Always require both confidentially **and** integrity.

But should we add confidentially first or integrity?

Different approaches:

- encrypt then MAC (e.g., in IPSec, use whenever possible)
- MAC then encrypt (e.g., in TLS)
- encrypt and MAC (e.g., in SSH)



Authenticated Encryption

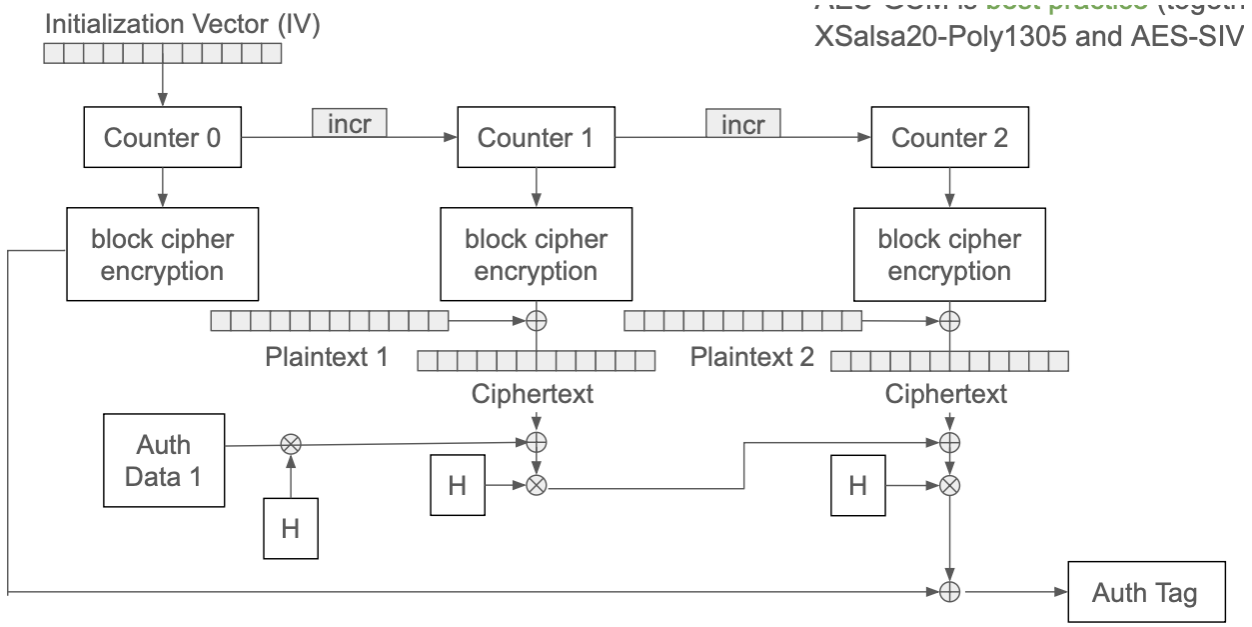
Modern encryption modes guarantee confidentiality and integrity

They include additional data that is authenticated but not encrypted: used for sequence numbers or other metadata

AEAD (Authenticated Encryption with Associated Data)

Example: Galois/Counter Mode GCM

Idea: We're adding a counter that enumerates the different blocks and simply increments the counter that have been used and feed the counter in addition to the block cipher.



Used with any block cipher, typically AES

AES-GCM is best practice (together with XSalsa20-Poly1305 and AES-SIV)

Public-key Cryptography

Solves the problem of having to agree on a pre-shared symmetric key

Uses a **pair** of public and private key instead

Encryption with the public key guarantees that only someone with the private key can decrypt.

Signing with the private key allows anyone with the public key to check validity.

The public key can be transferred **freely**, no need to keep it secret

Much better than symmetric keys!

Origin of Public-Key Crypto

1975 Diffie and Hellman describe the idea of asymmetric crypto in "New directions in Cryptography".
"...minimize the necessity of secure key distribution channels and supply the equivalent of a written signature...

Original Primitives

Public and private key

- Two keys: public is widely distributed, private is kept secret
- Must be hard to derive private from public
- May be easy to derive public from private

Operation: Encryption and decryption

- Encrypt with public key, decrypt with private key
- Hard to decrypt without private key

Operation: Digital signature

- Sign with private key, verify with public key
- Hard to create signature without private key

Interactive key exchange

- Establish a shared secret key over an insecure communication channel

Digital Signatures

Private key allows signer to generate an unforgeable signature that attests to the validity of message

Anybody can **verify** the signature using the public key

Not just encrypting with the secret key

Example signature schemes:

- ECDSA and EdDSA (elliptic curve based, most common)
- BLS (pairing based, aggregatable, should be used more imo)
- DLithium (lattice-based), SPHINCS+ (hard-based). Post Quantum!

Usage example: software-update distribution

- Your computer knows the public key of the software developer
  - type apt-key list on Debian/Ubuntu to see them
- Updates are signed
- Your computer can verify that the update indeed comes from the developer and that is has not been altered on its way

Usage example: authenticated email

- If Alice knows Bob's public key, she can verify Bob's signature on messages

Usage example: Contacts

- In Switzerland, you can sign contracts with digital signatures

Diffie-Hellman Key Exchange

Alice and Bob want to exchange data securely. Suppose Eve can listen to any messages in our scenario.

Solution 1: Alice and Bob agree on key ahead of time (this is the symmetric key from earlier). The difficulty comes from agreement of the key. How can we make this without having to agree on the key?

Solution 2: Multiple round-trips. Alice and Bob exchange multiple messages. Alice locks the box with her lock. Sends it over and Bob locks the box with his lock and sends it back. Alice removes her lock and etc...

Diffie-Helmman Key Exchange

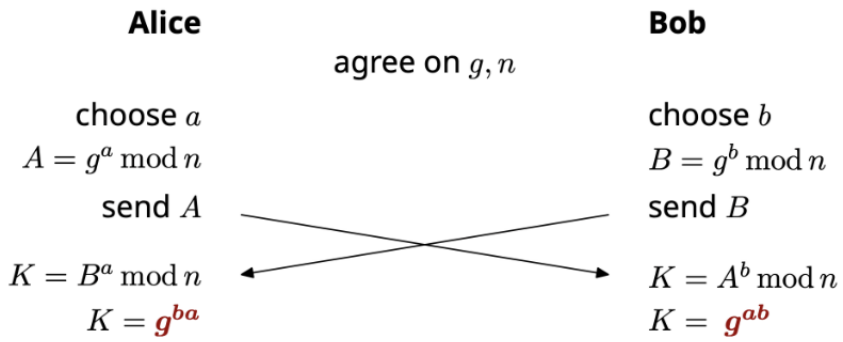


Figure 3: By exchanging the public values A, B, and combining them with their private values b, a, Alice Bob can create a shared secret K

An attacker who observes A and B cannot find K

Security Properties

The reverse function of $A = g^a \% p$ is difficult (hard) due to discrete logarithm. Of course, numbers p and g but be chosen well (large enough). Standard usually define p and g

Problem: Powerful man in the middle attack. Mallory (malicious attacker) can do a Diffie-Hellman exchange with both Alice and Bob, thus having both keys to communicate with both of them.

Man in the middle may break Diffie-Hellman

Must authenticate each other using, e.g., public keys

RSA Encryption

Goals:

- Alice wants to receive messages from any sender (confidentiality)
- Alice wants to prove message authorship to any receiver (integrity/authenticity)

Public key cryptography setup:

- Alice generates (sk, pk)
- Alice distributes pk, keeps sk secret

RSA:

- Pick primes p, q, computer $\mathbf{n = pq}$
- Compute $\phi(n) = (p-1)(q-1)$
- Pick **e** such that $1 < e < \phi(n)$ and $gcd(\phi(n), e) = 1$

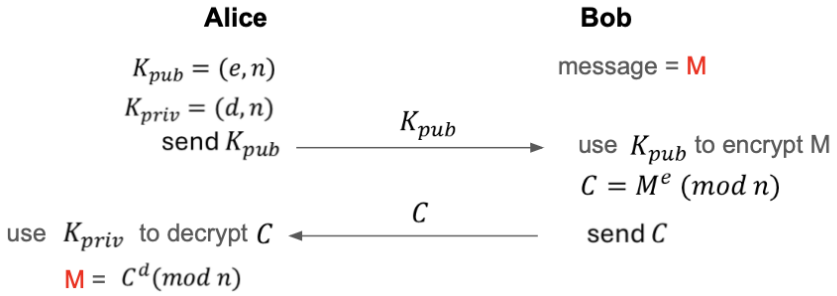
- In practice, use small prime $\mathbf{e} = 2^{16} + 1$
- Find **d** such that $e \cdot d \equiv 1 \ mod \ \phi(n)$. In other words, **d** is the modular inverse of **e**, $(d \equiv e^{-1} \ mod \ \phi(n))$
- Key-pair: **sk = (d, n), pk = (e, n)**

Encryption and decryption

- $Enc_{pk}(m) = m^e$ mod n
- $Dec_{sk}(c) = c^d$ mod n

Signing and verification

- $Sign_{sk}(m) = m^d$ mod n
- $Verify_{pk}(m, s) =$ (m == s^e mod n)



But we usually want to sign large messages

- RSA is expensive
- Solution: hash the message then sign the hash!

RSA can be broken by factoring n. Use a number of 3072 bits or more (for 128 bits of security) Scheme above is trivially insecure. Use a padding scheme (preferably RSA-OAEP)

(Lab): Don't use textbook RSA

- Any **deterministic scheme** is not IND-CPA (Indistinguishably under chosen plaintext attack). This is the weakest notion of provable security that one would hope for in an encryption scheme, which is why in practice we are usually adding some randomized padding scheme on top of the textbook RSA. The attack works by simply outputting 0 if the **LoR(m, m)** to get an encryption of a message **m**. Then call **LoR(m, m')** with **m'** $\neq$ **m**, if the result is the same as the previous, return 0, else return 1. OR an simpler attack can work by observing that **0** to any power is **0** and **1** to any power is **1**.
- RSA is **malleable**. It satisfies multiplicative homomorphism: $(m_1 m_2)^e \equiv m_1^e m_2^e \ mod \ n$. ($e$ is the public key) So even though you cannot call decrypt to the original ciphertext c, you can construct any new related ciphertext such as $c' = c \cdot t^e \ mod \ n$ for some t, then call decrypt on c', then you can recover the plaintext for c by dividing this by t (modulo n).

(Lab): Hastad's broadcast attack for RSA with low public exponent.

Chinese Remainder Theorem

If the n_i are pairwise coprime, and if $a_1, ..., a_k$ are any integers, then the system:

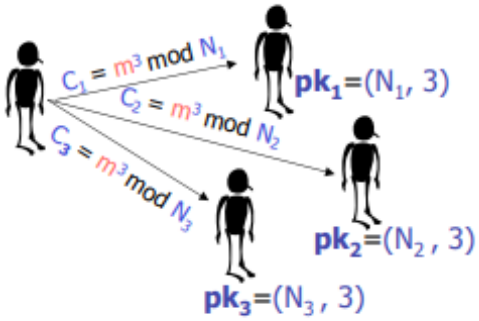
$x \equiv a_1 \dot{(}mod\ n_1)$
...
$x \equiv a_k \dot{(}mod\ n_k)$

Has a solution, and any two solutions, say x_1 and x_2, are congruent modulo N, that is, $x_1 \equiv x_2\ (mod)$.

If N_1, N_2, N_3 are relatively prime then by Chinese Remainder Theorem can combine

$C_1 = m^3\ mod\ N_1$
$C_2 = m^3\ mod\ N_2$
$C_3 = m^3\ mod\ N_3$

To find $C = m^3\ mod\ N_1\ N_2\ N_3$.
Since $m^3 < N_1\ N_2\ N_3$, thus $m \Leftarrow \sqrt[3]{C}$



Elliptic Curve Cryptography (ECC)

Based on elliptic curves over finite fields

Used everywhere, e.g., can adapt Diffie-Hellman with different group operations

Smaller keys for **equivalent** security (e.g., 256 bit ECC comparable to 3072 bit RSA)

Popular curves offer fast performance:

- curve25519
- Curve448
- NIST P256 (controversial)

ECC is used for key exchange, e.g., in TLS: ECC Diffie-Hellman (ECDHE)

Used for signature: ECC Digital Signature Algorithm (ECDSA)

For example, Sony used ECDSA to sign the software of Playstation 3

- ECDSA uses the private key for a nonce to gerate the signature
- The private key can be recovered if the same nonce is used in two signatures
- Easy to misuse, EdDSA improves on this (deterministic nonge generation)

Bitcoin addresses are a hash of an ECDSA public key

iMessages and iCloud keychan sync rely on ECDSA

Crypto Comparison

Asymmetric is powerful but orders of magnitude slower than symmetric crypto

Asymmetric used to exchange a symmetric key, then symmetric key later.

All these algorithms are only safe with long enough keys (for 128 bits of security)

- Symmetric: 128 to 256 bits
- Asymmetric: RSA 3072 bits, ECC 256 bits
- Hash functions: 256 bits

Cryptography is used in real life:

- Symmetric: Kerberos
- Asymmetric: WPA3 (not possible to crack WiFi password), SRP

Public-key Infrastructure (PKI)

Key Distribution

Cryptography reduces a security problem to one of key management

Public keys do not have to be secret, but still must be **authentic**

If a man in the middle can replace a public key, security guarantees are lost

We need a trusted third party to distribute the public keys

The Certification Authority (**CA**) certifies the keys by signing them. It does some validation before signing the keys

- may ask for an email
- may ask for a copy of your passport
- this is documented in the Certificate Practice Statement (**CPS**) of the CA)

If we trust the key of the CA, we can trust all keys signed by the CA. A 'signed key' is a **certificate**. It contains at least:

- the identity of the holder (subject)
- the validity date of the certificate
- the public key of the subject
- the signature by the CA

Hierarchy of Trust

Current browsers know a set of root CAs (144 root CAs in Firefox as of 11/2023). If there is a chain of signatures up to a trusted root, the browser trusts the certificate.

Case Study: TLS

TLS

Transport Layer Security (TLS, previously know as SSL) provides confidentiality, integrity, and authentication

"The primary goal of TLS is to provide a secure channel between two communicating peers"

Basic idea: build your client-server app without security, add TLS, et voilà!

TLS Building Blocks

The server is authenticated with a certificate.

It proves its identity by signing some information received from the client with its private key.

Client and server create a symmetric key using asymmetric crypto

- EC Diffie Hellman **only** (since 1.3)
- Previously also supported standard Diffie-Hellman and key transfer

Once key established, used an AEAD to communicate (Authenticated Encryption with Associated Data)

TLS Cipher Suites (after 1.3)

Algorithms to be used are specified in cipher suites: TLS_AES_128_GCM_SHA_256

Key exchange and signatures default to ECDHE and RSA

Lesson from previous versions of the protocol:

- Flexibility in cryptography can actually **hurt** security
- Better to stick to **same defaults**.

Simplicity is key!

TLS 1.3

Removes old, no-longer-safe cipher suites, remove compression

Remaining cipher suites all use AEAD

Handshake

- can be shorter in some cases
- is partially encrypted

Key exchange is always forward secure

Session resumption with pre-shared key (PSK)

About 63% of website offer TLS 1.3 now

Perfect Forward Secrecy

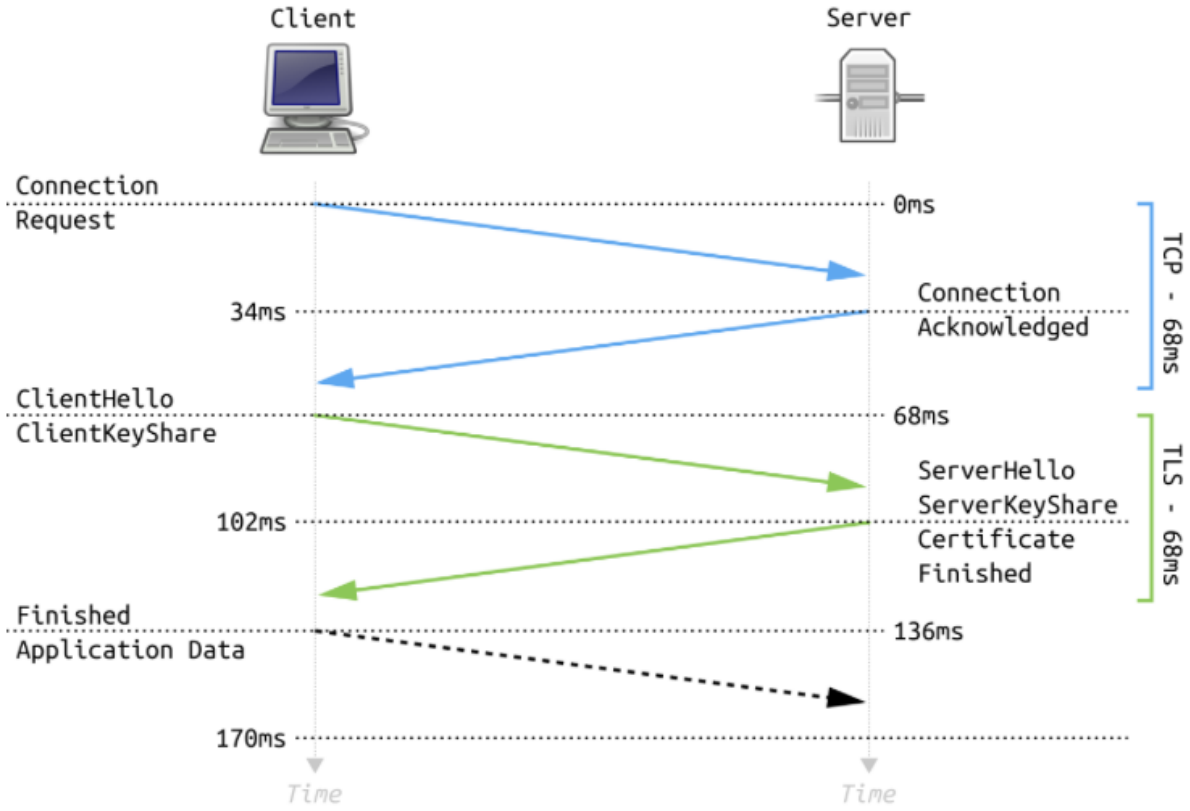
Strong security notion: session keys should not be compromised even if long-term secrets are.

Diffie-Hellman-like protocol offers Perfect Forward Secrecy (PFS)

At least since Ed. Snowden's revelations we know that:

- Governments record a lot of traffic
- Governments exploit opportunities to steal private keys

TLS Handshake



TLS Uses Public and Symmetric Crypto

Public key crypto:

- The CA has signed the server's certificate (and its own root certificate)
- The server signs the key exchange to prove it hold the private key
- Elliptic curve Diffie Hellman is used to exchange a symmetric key

Symmetric crypto:

- AES block cipher is used in CBC mode for encryption
- SHA has is used for HMAC, for key derivations

Implementing TLS

Two major ways of implementing TLS

Use a new name and port (HTTPS, LDAPS, IMAPS, etc.):

- HTTP on port 80 $\Rightarrow$ HTTPS on 443

- Start with a TLS handshake, security is mandatory
- Not compatible with clients that can't TLS

Use the STARTTLS command on the standard protocol

- e.g., ESMTP (extended SMTP) on port 25
- client types STARTTLES if it want to start a handshake
- Opportunistic encryption, no guarantees
- MITM can pretend STARTTLS is not supported

Deploying TLS on the Internet

Increasing Usage of TLS

Using HTTPS everywhere would be better for Privacy and Security. This is quite recent, HTTP is being down-ranked and HTTPS is being preferred.

Let's Encrypt: Free Certificates

To able to create certificates that are trusted by all browser, you must undergo a costly certification

- Prove that you protect your private keys
- Prove that you diligently validate the identity of your customers etc.

The Internet Security Research Group (not for profit) found enough sponsors to certify a fully automated CA that gives certificates for free!

It's called **Let's Encrypt**

To obtain a certificate, you must place specific data

- in a file on your web server, or
- in a DNS entry of your domain

Fully automatable: **no excuse for not using TLS!**

Attacks on HTTPS and Defenses

SSL Stripping and HSTS

A MITM makes you believe that the site uses HTTP, not HTTPS

- When you type a URL, your browser first connects using HTTP
  - The server sends a redirect to HTTPS

  - The MITM doesn't show you the redirect, your browser continues to use HTTP
- Or, attacker replaces HTTPS with HTTP in the link of the pages that you visit

HSTS

HTTP Strict Transport Security (HSTS) RFC 6797

The server sends an HTTP header indicating you must always use HTTPS.

Strict-Transport-Security: max-age=63072000; includeSubDomains;

Where '63072000' is one year.

Can you spot the weakness?

If the MITM intercepts your very first connection to the site, they can hide the redirection to HTTPS
- The client will **never see** the HSTS header

You can request that your domain be added to the HSTS-**preload** list

E.g., digitec.ch is on the list, your browser will never try HTTP when connecting to digitec

Untrustworthy CAs

A trusted CA (the root cert is in your browser) can give the MITM a trusted certificate in the name of **any** server

Using this, the server can intercept the traffic without you knowing.

Reasons for CA To hand out face certs:

- The CA has been hacked (2021 Comodo was hacked, certificates were generate for, e.g., www.google.com, login.yahoo.com, login.skype.com)
- The CA is your company's CA. Companies often install their root certificate authorities so that they can MITM all your traffic for malware protection for it's employees
- The government may have requested the certs in order to spy on its citizens (Syria, China, India, or France)

Protection Against Untrustworthy CAs

- **Certificate pinning**: client-side list of trusted certificates
  - The developer of the client application stores the certificate of a trusted root CA, or intermediate CA in the client
  - If the server show a certificate which is signed by this **pin**, it does not accept to connect, e.g., a mobile e-banking application only trusts certificates signed by an intermediate CA of the bank
- **Certificate transparency**: public list of certificates
  - Public signed logs of certs used in the internet. CAs submit all the certs they generate

- Servers can verify if the logs contain certs that they did not request. Then, somebody else requested a cert for their domain!

this is a green bold text